处理文本文件

文件类型

- 文件就是字节流（byte stream）或叫字符流
- 文本文件，即字符流文件，如：text。需要指定字符编码：ASCII、Unicode、GB、UTF8 .....
- 二进制格式文件，即非文本文件，如：图片文件、影音文件。
- python可以很容易读取文本文件。优点：跨平台、易于使用。

文件的打开

- 文件读取和写入之前要打开。
- open()的第二个参数是打开方式，”r“是读取（read）的意思。以读取方式打开一个文件前，应测试其是否存在。

```
import os
if os.path.isfile("read_it.txt"):
    text_file = open("read_it.txt", "r")
```

文件指针（游标）

- file.tell()去检查当前文件指针的位置
- file.seek(0)回到文件第0处。即文件头部
- file.seek()方法标准格式是：

  `seek(offset,[whence=0])`

  - offset：相对于whence的偏移量，字节数，可以为负数，whence省略时，默认从文件开头算起
  - whence：给offset参数的偏移参考，表示要从哪个位置开始偏移；0代表从文件开头开始算起，1代表从当前位置开始算起，2代表从文件末尾算起。

换行符

- 不同的操作系统有不同的方案。
  - Unix/Linux 中使用一个换行符 LF(0x0A) 表示换行 ('\n')
  - 苹果MAC OS中使用一个回车符 CR(0x0D) 表示下一行 ('\r')
  - Windows 中则使用0x0D和0x0A两个字符 ("\r\n")，。
- 可以使用`os.linesep`查询当前操作系统使用的换行符。

文件的读取

- read()：从游标处开始一直读取到文件尾
- read(n)：从游标处开始读取n个字符
- readline()：从游标处开始读取一行，即遇到\n停止。
- readline(n)：从游标处开始读取n个字符，如果遇到\n，也会停止。

- readlines()：读取文件中的所有行，并以列表的形式返回。
- 文本文件：animals.txt

```
Ant Bee
Cat Dog
Eel Fox
```

```
infile = open("animals.txt","r")
print("(1) Using read(): ")
print(infile.read())
infile.close()
```

```
(1) Using read():
Ant Bee
Cat Dog
Eel Fox
```

```
infile = open("animals.txt","r")
print("(2) Using read(number): ")
s1 = infile.read(4)
print(s1)
s2 = infile.read(10)
print(repr(s2))
infile.close()
```

```
(2) Using read(number):
Ant
'Bee\nCat Do'
```

```
Ant Bee\nCat Dog\nEel Fox\n
^ 文件指针初始化
Ant Bee\nCat Dog\nEel Fox\n
    ^ 执行 s1 = infile.read(4) 之后 s1 是 'Ant '
Ant Bee\nCat Dog\nEel Fox\n
               ^ 执行 s2 = infile.read(10) 之后 s2 是 'Bee\nCat Do'
```

```
infile = open("animals.txt","r")
print("(3) Using readline(): ")
line1 = infile.readline()
line2 = infile.readline()
line3 = infile.readline()
line4 = infile.readline()
print(repr(line1))
print(repr(line2))
print(repr(line3))
print(repr(line4))
infile.close()
```

```
(3)Using readline():
'Ant Bee\n'
'Cat Dog\n'
'Eel Fox\n'
''
```

```
Ant Bee\nCat Dog\nEel Fox\n
^ 文件指针初始化，指向 animals.txt 文件头部
Ant Bee\nCat Dog\nEel Fox\n
         ^ 执行 line1 = infile.readline() 之后 line1 是 'Ant Bee\n'
Ant Bee\nCat Dog\nEel Fox\n
                  ^ 执行 line2 = infile.readline() 之后 line2 是 'Cat Dog\n'
Ant Bee\nCat Dog\nEel Fox\n
                            ^ 执行 line3 = infile.readline() 之后 line3 是 'Eel Fox\n'
Ant Bee\nCat Dog\nEel Fox\n
                            ^ 执行 line4 = infile.readline() 之后 line4 是 ''
```

```
infile = open("animals.txt","r")
print("(4) Using readlines(): ")
print(infile.readlines())
infile.close()
```

```
(4) Using readlines():
['Ant Bee\n', 'Cat Dog\n', 'Eel Fox\n']
```

文件的关闭

- text_file代表打开了的文件
- 文件使用完后要关闭
- `text_file.close()`

打开文件的读写模式

- r：以只读方式打开文件（默认模式）。文件指针定位在文件头的位置。如果文件不存在会报错。
- w：以只写方式打开文件。如果文件存在，则打开文件，清空文件内容，从文件头开始编辑；如果文件不存在，则创建新文件，打开编辑。
- a：以追加方式打开文件，同样是只写，不允许进行读操作。如果文件存在，则打开文件，将文件指针定位到文件尾。因此，新的内容是追加在已有内容之后。如果文件不存在，则创建新文件进行写入。

字符串写入文件

- 需要先以写方式打开文件。
- 如果该文件已经存在就会被覆盖。
- write() 方法需要手工在行末加入换行符“\n”。
- 最后记得关闭文件。

```
# WriteDemo.py
# Open file for output
outfile = open("animals.txt", "w")

# Write data to the file
outfile.write("Ant Bee\n")
outfile.write("Cat Dog\n")
outfile.write("Eel Fox\n")

outfile.close() # Close the output file
```

```
^ 文件指针初始化，指向空文件 animals.txt 头部
Ant Bee\n
         ^ 执行 outfile.write("Ant Bee\n") 后
Ant Bee\nCat Dog\n
                  ^ 执行 outfile.write("Cat Dog\n") 后
Ant Bee\nCat Dog\nEel Fox\n
                           ^ 执行 outfile.write("Eel Fox\n") 后
```

字符串列表写入文件

- 使用 writelines() 方法

```
text_file = open("write_it.txt", "w")
lines = ["Line 1\n",
"This is line 2\n",
"That makes this line 3\n"]
text_file.writelines(lines)
text_file.close()
```

追加数据

- 使用追加方式 a 打开文件

```
# Open file for appending data
outfile = open("Info.txt", "a")
outfile.write("Python is interpreted\n")
outfile.close() # Close the input file
```

```
Programming is fun\n
                    ^ 文件指针初始化，指向 Info.txt 文件尾。
Programming is fun\nPython is interpreted\n
                                          ^ 执行 outfile.write("Python is interpreted\n") 后
```

转成字符串后再存入

```
# WriteReadNumbers.py
from random import randint
```

```
def main():
    # Open file for writing data
    outfile = open("Numbers.txt", "w")
    for i in range(10):
        outfile.write(str(randint(0, 9)) + " ")
    outfile.close() # Close the file

    # Open file for reading data
    infile = open("Numbers.txt", "r")
    s = infile.read()
    numbers = [eval(x) for x in s.split()]
    for number in numbers:
        print(number, end = " ")
    infile.close() # Close the file

main() # Call the main function
```

处理大文件

- 可以用 while 循环

```
line = inline.readline()
while line != '':
    line = inline.readline()
    # process the line here...
```

- 也可以用 for 循环，更加简洁

```
for line in inline:
    # process the line here...
```

- 一个复制文本文件的例子

```
# CopyFile.py
import os.path
import sys

def main():
    # Prompt the user to enter filenames
    f1 = input("Enter a source file: ").strip()
    f2 = input("Enter a target file: ").strip()

    # Check if target file exists
    if os.path.isfile(f2):
        print(f2 + " already exists")
        sys.exit()

    # Open files for input and output
    infile = open(f1, "r")
```

```
    outfile = open(f2, "w")

    # Copy from input file to output file
    countLines = countChars = 0
    for line in infile:
        countLines += 1
        countChars += len(line)
        outfile.write(line)
    print(countLines, "lines and", countChars, "chars copied")

    infile.close()  # Close the input file
    outfile.close() # Close the output file

main() # Call the main function
```

打开文件的组合模式

- r+：打开一个文件用于读写。如果文件存在，则打开文件，将文件指针定位在文件头，新写入的内容在原有内容的前面；如果文件不存在会报错。
- w+：打开一个文件用于读写。如果文件存在，则打开文件，清空原有内容，进入编辑模式；如果文件不存在，则创建一个新文件进行读写操作。
- a+：以追加模式打开一个文件用于读写。如果文件存在，则打开文件，将文件指针定位在文件尾，新写入的内容在原有内容的后面；如果文件不存在，则创建一个新文件用于读写。
- +：打开一个文件进行更新（可读写）。注意：该模式不能单独使用，需要与r/w/a组合使用。文件指针的位置取决于另一个组合参数。
- 所有上面这些模式默认都是t——文本模式，如果要以二进制模式打开，需要加上参数b，如：rb、rb+、wb、wb+、ab、ab+。

文本文件访问模式小结

模式	文本文件访问模式
'r'	读取文件。以只读方式打开文件（默认模式）。文件指针定位在文件头的位置。如果文件不存在会报错。
'w'	写入文件。以只写方式打开文件。如果文件存在，则打开文件，清空文件内容，从文件头开始编辑；如果文件不存在，则创建新文件，打开编辑。
'a'	向文件中添加内容。以追加方式打开文件，同样是只写，不允许进行读操作。如果文件存在，则打开文件，将文件指针定位到文件尾。如果文件不存在，则创建新文件进行写入。
'r+'	读写文件。文件指针定位在文件头的位置。如果文件不存在会报错。
'w+'	读写文件。如果文件存在，则打开文件，清空文件内容，从文件头开始编辑；如果文件不存在，则创建新文件，打开编辑。
'a+'	读取文件以及向文件中添加内容。如果文件存在，则打开文件，将文件指针定位到文件尾。如果文件不存在，则创建新文件进行写入。

常用文件函数小结

函数名	说明
file=open(filename,mode)	按照指定的模式打开文件
file.close()	关闭文件。已关闭的文件不能被读取或写入，除非再次将其打开
file.read([size])	从文件中读取 size 个字符，并将结果以字符串的形式返回。如果没有指定 size，该方法就会返回从当前位置到文件末尾的全部字符
file.readline([size])	从文件的当前行中读取 size 个字符，并将结果以字符串的形式返回。如果没有指定 size，该方法就会返回从当前位置到该行末尾的全部字符
file.readlines()	读取文件中的所有行，并以列表的形式返回
file.write(output)	将字符串 output 写入到文件
file.writelines(output)	将字符串列表 output 写入到文件

使用with语句

Python 中的 **with** 语句用于异常处理，封装了 **try...except...finally** 编码范式，提高了易用性。**with** 语句使代码更清晰、更具可读性， 它简化了文件流等公共资源的管理。在处理文件对象时使用 with 关键字是一种很好的做法。

我们可以看下以下几种代码实例。

- 不使用 **with**，也不使用 **try...except...finally** 的实例

```
file = open('./test.txt', 'w')
file.write('hello world !')
file.close()
```

以上代码如果在调用 write 的过程中，出现了异常，则 close 方法将无法被执行，因此资源就会一直被该程序占用而无法被释放。

- 使用 **try...except...finally** 来改进代码的实例

```
file = open('./test.txt', 'w')
try:
    file.write('hello world')
finally:
    file.close()
```

以上代码我们对可能发生异常的代码处进行 try 捕获，发生异常时执行 except 代码块，finally 代码块是无论什么情况都会执行，所以文件会被关闭，不会因为执行异常而占用资源。

- 使用 **with** 关键字的实例

```
with open('./test.txt', 'w') as file:
    file.write('hello world !')
```

使用 **with** 关键字系统会自动调用 f.close() 方法， with 的作用等效于 try/finally 语句是一样的。我们可以在执行 with 关键字后检验文件是否关闭

```
print(file.closed) # True
```

with 语句实现原理建立在上下文管理器之上。上下文管理器是一个实现 **enter** 和 **exit** 方法的类。使用 with 语句确保在嵌套块的末尾调用 **exit** 方法。这个概念类似于 try...finally 块的使用。

在文件对象中定义了 **enter** 和 **exit** 方法，即文件对象也实现了上下文管理器，首先调用 **enter** 方法，然后执行 with 语句中的代码，最后调用 **exit** 方法。 即使出现错误，也会调用 **exit** 方法，也就是会关闭文件流。

并不是所有的对象都支持 with 语句，只有支持上下文管理协议的对象才能够使用 with 语句。

CSV格式文件

使用 csv 模块处理 CSV 文件

CSV 文件是简单的，缺少 Excel 电子表格的许多功能。例如，CSV 文件中：

- 值没有类型，所有东西都是字符串；
- 没有字体大小或颜色的设置；
- 没有多个工作表；
- 不能指定单元格的宽度和高度；
- 不能合并单元格；
- 不能嵌入图像或图表。

CSV 的文件的优势是简单。CSV 文件被许多种类的程序广泛地支持，可以在文本编辑器中查看，它是表示电子表格数据的直接方式。CSV 格式和它声称的完全一致：它就是一个文本文件，具有逗号分隔的值。

因为 CSV 文件就是文本文件，所以你可能会尝试将它们读入一个字符串，例如，因为 CSV 文件中的每个单元格有逗号分割，也许你可以只是对每行文本调用 split()方法，来取得这些值。但并非CSV 文件中的每个逗号，都表示两个单元格之间的分界。CSV 文件也有自己的转义字符，允许逗号和其他字符作为值的一部分。split()方法不能处理这些转义字符。因为这些潜在的缺陷，所以应该使用 csv 模块来读写 CSV 文件。csv 模块是 Python 自带的，所以不需要安装就可以导入它。

Reader 对象

要用 csv 模块从 CSV 文件中读取数据，需要创建一个 Reader 对象。Reader 对象让你迭代遍历 CSV 文件中的每一行。数据文件example.csv 的内容如下：

```
4/5/2015 13:34,Apples,73
4/5/2015 3:41,Cherries,85
4/6/2015 12:46,Pears,14
4/8/2015 8:59,Oranges,52
4/10/2015 2:07,Apples,152
4/10/2015 18:10,Bananas,23
4/10/2015 2:40,Strawberries,98
```

```
import csv
exampleFile = open('example.csv')
exampleReader = csv.reader(exampleFile)
exampleData = list(exampleReader)
print(exampleData)
# [['4/5/2015 13:34', 'Apples', '73'], ['4/5/2015 3:41', 'Cherries', '85'], ['4/6/2015
12:46', 'Pears', '14'], ['4/8/2015 8:59', 'Oranges', '52'], ['4/10/2015 2:07', 'Apples',
'152'], ['4/10/2015 18:10', 'Bananas', '23'], ['4/10/2015 2:40', 'Strawberries', '98']]
```

既然已经将 CSV 文件表示为列表的列表，就可以用表达式 `exampleData[row][col]` 来访问特定行和列的值。其中，row 是 exampleData 中一个列表的下标，col 是该列表中你想访问的项的下标。

```
print(exampleData[0][0])
# '4/5/2015 13:34'
print(exampleData[0][1])
#  'Apples'
print(exampleData[0][2])
#  '73'
print(exampleData[1][1])
#  'Cherries'
print(exampleData[6][1])
#  'Strawberries'
```

对于大型的 CSV 文件，你需要在一个 for 循环中使用 Reader 对象。这样避免将整个文件一次性装入内存。例如

```
import csv
exampleFile = open('example.csv')
exampleReader = csv.reader(exampleFile)
for row in exampleReader:
    print('Row #' + str(exampleReader.line_num) + ' ' + str(row))

'''
Row #1 ['4/5/2015 13:34', 'Apples', '73']
Row #2 ['4/5/2015 3:41', 'Cherries', '85']
Row #3 ['4/6/2015 12:46', 'Pears', '14']
Row #4 ['4/8/2015 8:59', 'Oranges', '52']
Row #5 ['4/10/2015 2:07', 'Apples', '152']
Row #6 ['4/10/2015 18:10', 'Bananas', '23']
Row #7 ['4/10/2015 2:40', 'Strawberries', '98']
'''
```

Writer 对象

Writer 对象让你将数据写入 CSV 文件。要创建一个 Writer 对象，就使用 csv.writer() 函数。

```
import csv
outputFile = open('output.csv', 'w', newline='')
outputWriter = csv.writer(outputFile)
outputWriter.writerow(['spam', 'eggs', 'bacon', 'ham'])
outputWriter.writerow(['Hello, world!', 'eggs', 'bacon', 'ham'])
outputWriter.writerow([1, 2, 3.141592, 4])
outputFile.close()
```

输出文件 output.csv 的内容如下：

```
spam,eggs,bacon,ham
"Hello, world!",eggs,bacon,ham
1,2,3.141592,4
```

delimiter 参数

假定你希望用制表符代替逗号来分隔单元格，则可以使用 delimiter 参数加以指定。例如，对于 TSV 文件：

```
import csv
exampleFile = open('example.tsv')
exampleReader = csv.reader(exampleFile, delimiter='\t')
exampleData = list(exampleReader)
print(exampleData)
exampleFile.close()

outputFile = open('output.tsv', 'w', newline='')
outputWriter = csv.writer(outputFile, delimiter='\t')
outputWriter.writerow(['spam', 'eggs', 'bacon', 'ham'])
outputWriter.writerow(['Hello, world!', 'eggs', 'bacon', 'ham'])
outputWriter.writerow([1, 2, 3.141592, 4])
outputFile.close()
```

文件目录操作

要导入的模块

- 使用内置模块 os，可提供操作系统的基本功能。需要首先导入：`import os`
- os.path 模块包含了许多与文件和目录相关的函数。因为 os.path 是 os 模块中的模块，所以也是使用 `import os` 进行导入。
- 使用内置模块 shutil，即 shell utilities , 可提供文件和目录的复制、移动、改名和删除功能。需要首先导入：`import shutil`

文件与目录的基本概念

文件名

- 文件名与目录名是否区分大小写（或称之为大小写是否敏感 case sensitive）。一般来说，Windows不区分大小写，Linux/UNIX/BSD区分大小写，而MacOS与Windows类似不区分大小写。

  > 是否区分大小写，会带来很多问题，从一个区分大小写的系统中复制文件到一个不区分大小写的系统时，可能会出现“**文件被覆盖**”的风险。例如，从Linux中复制某一目录到Windows时，如果该目录下同时有 "file1" 和 "File1" 二个文件，在Linux中，这两个文件会被视为二个不同的文件而和平共处，而在Windows中会认为这二个文件是同一个文件，后复制来的文件将会覆盖先复制来的文件。

- 隐藏文件。Linux/UNIX/BSD/MacOS等操作系统中，以 `.` 开头的文件会被认为是隐藏文件，如 `.file1`。

目录

- 目录（directory，也称为文件夹 folder ）。目录中可以包含文件和其他子目录，目录以树状形式组织起来，称为目录树（directory tree）。一个文件系统（filesystem）一般包含且仅包含一个根目录。

  > 注意Linux/UNIX/BSD/MacOS 等系统的运行环境下只有一个（虚拟）根目录，而Windows一般采用字母作为盘符（如 `C:`）来标识不同的文件系统，这使得冒号（colon）在Linux/UNIX/BSD/MacOS等系统中可用于文件名和目录名中，但是在Windows中冒号（colon）是不允许作为文件名或目录名的一部分的。在不同平台批量复制文件到Windows时可能会因某文件名或目录名中包含冒号（colon）而报错。

- 目录之间的分隔符：Windows使用反斜杠（backslash），而Linux/UNIX/BSD/MacOS等使用正斜杠（slash）。在 Python 中可以使用 `os.sep` 得到当前操作系统所使用的目录分隔符。

  > 正斜杠（slash）在Linux/UNIX/BSD/MacOS中不允许作为文件名或目录名的一部分，而反斜杠（backslash）在Windows中不允许作为文件名或目录名的一部分。在不同平台间批量复制文件时，可能会因某文件名或目录名中包含目标平台中的非法字符（目录分隔符）而报错。

  ```
  >>> import os
  >>> os.sep
  '/'
  ```

- Python可以使用 `os.path.join(paths)` 合并目录，并处理不同平台上目录分隔符的差异。

  ```
  >>> import os
  >>> os.path.join('usr','bin','spam')
  usr/bin/spam
  ```

  在 Linux/UNIX/BSD/MacOS 等平台上，会得到 `usr/bin/spam`。而在Windows平台上，会得到 `usr\\bin\\spam`。注意得到的字符串最前面并没有目录分隔符。

  > 注意：反斜杠（backslash）在Python中代表转义，所以如果在Python程序中的字符串中使用反斜杠，必须连写两遍，即 `\\`。

- 当前工作目录（current working directory，简称为当前目录）：每个运行中的程序，都有一个当前工作目录，所有没有从根目录开始标识的文件名或路径（即相对路径），都假定在当前工作目录中，或从当前工作目录开始查找。

- 使用 `os.getcwd()` 取得当前工作目录，使用 `os.chdir(path)` 改变当前工作目录。

```
>>> import os
>>> os.getcwd()
'/home/user1'
>>> os.chdir('/tmp')
>>> os.getcwd()
'/tmp'
```

- 特殊目录：
  - 根目录。即 `'/'` 或 `'\\'`。
  - 点（.）目录，点目录代表“这个目录本身”的意思。如果当前目录是 `/home/user1`，则 `./docs/file1.txt` 代表 `/home/user1/docs/file1.txt`。
  - 点点（..）目录，点点目录代表“这个目录的父目录”的意思。如果当前目录是 `/home/user1`，则 `../user2/proj/file1.txt` 代表 `/home/user2/proj/file1.txt`。
- 绝对路径与相对路径。
  - 绝对路径（absolute path），指从根目录开始标识的路径，如 `/home/user1/docs/file1.txt`。
  - 相对路径（relative文件和目录的 path），指不从根目录开始标识的路径，如 `docs/file1.txt`。相对路径总是从当前目录开始进行查找。如果当前目录是 `/home/user1`，那么这个文件是完整路径为 `/home/user1/docs/file1.txt`。
  - 使用 `os.path.abspath(path)` 将返回参数path的绝对路径。如果path是一个相对路径，则在此相对路径前补充上当前目录。如果path本来就是绝对路径，则直接原样输出。

    ```
    >>> import os
    >>> os.getcwd()
    '/home/user1'
    >>> os.path.abspath('docs/doc1')
    '/home/user1/docs/doc1'
    >>> os.path.abspath('.')
    '/home/user1'
    >>> os.path.abspath('../user2/file1')
    '/home/user2/file1'
    >>> os.path.abspath('/dir1')
    '/dir1'
    ```

  - 使用 `os.path.relpath(path,start)` 将返回从start路径到path的相对路径。如果没有提供start，就使用当前工作目录作为开始路径。如果path本来就是相对路径，则直接原样输出。

```
>>> import os
>>> os.getcwd()
'/home/user1'
>>> os.path.relpath("/a")
'../../a'
>>> os.path.relpath("/a","/")
'a'
>>> os.path.relpath("/home/user1/a")
'a'
>>> os.path.relpath('a')
'a'
```

  - 使用 `os.path.isabs(path)` 判断一个路径是否为绝对路径。是绝对路径就返回True，是相对路径就返回False。

```
>>> import os
>>> os.path.isabs(".")
False
>>> os.path.isabs('/dir1')
True
```

- 切分路径。
  - 使用 `os.path.dirname(path)` 返回path参数中最后一个目录分隔符（斜杠）之前的所有内容。
  - 使用 `os.path.basename(path)` 返回path参数中最后一个目录分隔符（斜杠）之后的所有内容。
  - 使用 `os.path.split(path)` 将path参数中最后一个目录分隔符（斜杠）为分界，分隔出二个字符串，放在一个元组中。
  - 如果需要进一步的切分路径，将path参数依所有的目录分隔符（斜杠）全部切分，可以使用字符串的split()方法，如 `path.split(os.path.sep)`，其中 `os.path.sep` 将依据所在操作系统平台生成正确的目录分隔符。

```
>>> import os
>>> path = '/home/user1/docs/file1'
>>> os.path.dirname(path)
'/home/user1/docs'
>>> os.path.basename(path)
'file1'
>>> os.path.split(path)
('/home/user1/docs', 'file1')
>>> path.split(os.path.sep)
['', 'home', 'user1', 'docs', 'file1']
```

查看目录内容和文件大小

- 使用 `os.listdir(path)` 可以返回目录中所有的内容。

```
>>> import os
>>> os.listdir('.')
['file1','file2','dir1','dir2']
```

- 使用 `os.path.getsize(file)` 可以得到文件的字节数。

```
>>> import os
>>> os.path.getsize('file1')
662
```

遍历目录树

- 使用 `os.walk(path)`，它将递归地遍历指定目录下所有的文件及目录，其返回的信息中，root表示正在浏览的目录，dirs表示该目录下所有的子目录，files表示该目录下所有的文件。

```
import os
for root, dirs, files in os.walk("."):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name))
```

检查路径的有效性

- 如果path参数所指的文件或目录存在，则 `os.path.exists(path)` 返回 True ， 否则返回 False 。
- 如果path参数所指的文件存在，则 `os.path.isfile(path)` 返回 True ， 否则返回 False 。
- 如果path参数所指的目录存在，则 `os.path.isdir(path)` 返回 True ， 否则返回 False 。

操作文件和目录

创建和删除目录

- 使用 `os.mkdir()` 创建新目录。此时要求新目录所在的目录已经存在。
- 使用 `os.makedirs()` 创建新目录 。如果需要的话，将创建所有必要的中间目录 ， 以确保完整路径。假如当前目录下没有 `src` 目录，下面的代码将创建 `src/main` 目录（假设本来不存在src目录）。

```
>>> import os
>>> os.makedirs("src/main")
```

- 删除空目录： `os.rmdir(path)` 。要求被删除的目录必须是空目录。
- 使用 `os.removedirs(path)` 可以删除多级目录。

复制文件和目录

- 复制文件：shutil.copy(source, destination)。返回值是新文件的路径。
    - 如果destination是一个目录，则将source复制到该目录下，文件名不变。
    - 如果destination不是一个目录，则将source复制一份，新文件名为destination。
    - 注意：如果目标地有同名文件，则会直接覆盖原文件。
    - source 必须为文件，否则会报错。

```
>>> import shutil
>>> shutil.copy('file1','file2') # 复制文件file1，新文件名为file2
>>> shutil.copy('file1','dir1') # 复制文件file1到目录dir1下，新文件名为dir1/file1
>>> shutil.copy('file1','dir1/file2') # 复制文件file1到目录dir1下，新文件名为dir1/file2
```

- 复制目录：shutil.copytree(source,destination)。返回值是新目录的路径。
  - 如果destination已存在，无论是文件还是目录，均报错。
  - 如果destination不存在，则将source目录复制一份，新目录名为destination。
  - source 必须为目录，否则会报错。

```
>>> import shutil
>>> shutil.copytree('dir1','dir2') # 复制目录dir1，新目录名为file2
>>> shutil.copytree('dir1','dir2/dir1') # 复制目录dir1到dir2下，新目录名为dir2/dir1
>>> shutil.copytree('dir1','dir2/dir3') # 复制目录dir1到dir2下，新目录名为dir2/dir3
```

文件和目录的移动与改名

- shutil.move(source,destination)。返回值是新文件/目录的路径。
  - source可以是文件或目录
  - 如果destination是一个目录，则将source移动到该目录下，原文件名/目录名不变。
  - 如果destination不是一个目录，则将source改名为destination。
  - 注意：如果source是文件，而目标地有同名文件，则会直接覆盖原文件。
  - 注意：如果source是目录，而目标地有同名文件，则会报错。

```
>>> import shutil
>>> shutil.move('file1','file5') # 将文件file1改名为file5
>>> shutil.move('dir1','dir5') # 将目录dir1改名为dir5
>>> shutil.move('file5','dir5') # 将文件file5移动到dir5目录下，仍叫file5
>>> shutil.move('file2','dir5/file6') # 将文件file2移动到dir5目录下，同时改名为file6
>>> shutil.move('dir2','dir5/dir6') # 将目录dir2移动到dir5目录下，同时改名为dir6
```

永久删除文件和目录

- 删除文件：`os.remove(path)` 或者 `os.unlink(path)`。要求被删除的是文件
- 删除任意目录：`shutil.rmtree(path)`。被删除的目录可以是非空目录

安全删除（移入回收站）

- 需要第三方模块。`pip install send2trash`
- send2trash(path)，被删除的可以是文件、空目录或非空目录，将被移动到回收站中。

```
>>> import send2trash
>>> send2trash.send2trash(file1)  # 文件 file1 被移动到回收站中
>>> send2trash.send2trash(dir1)   # 空目录 dir1 被移动到回收站中
>>> send2trash.send2trash(dir2)   # 非空目录 dir2 被移动到回收站中
```

快速检索

常量名/函数名	作用	shell类似命令
os.name	当前操作系统类型	
os.sep , os.path.sep	当前操作系统的路径分隔符	
os.linesep	当前操作系统的换行符	
os.getcwd()	获得当前目录	pwd
os.chdir()	切换当前目录	cd
os.path.join()	拼接完整路径	
os.path.abspath()	转为绝对路径	
os.path.relpath()	转为相对路径	
os.path.isabs()	判断是否为绝对路径	
os.path.dirname()	从一个路径中提取所在目录名	
os.path.basename()	从一个路径中提取文件名	
os.path.split()	获得所在目录和文件名	
os.path.splitext()	分离文件名和扩展名	
os.listdir()	列出一个目录下的内容	ls
os.walk()	递归遍历目录	ls -R
os.path.getsize()	获得一个文件的大小	
os.path.exists()	判断文件或目录是否存在	
os.path.isfile()	判断是否为一个文件	
os.path.isdir()	判断是否为一个目录	
os.mkdir()	创建目录	mkdir
os.makedirs()	创建多个目录	mkdir -p
shutil.copy()	复制文件	cp
shutil.copytree()	复制目录	cp -R
shutil.move()	文件目录移动或改名	mv
os.rename()	文件目录移动或改名	mv
os.remove()	删除文件	rm
os.unlink()	删除文件	rm

常量名/函数名	作用	shell类似命令
os.rmdir()	删除空目录	rmdir
os.removedirs()	删除多级空目录	rmdir -p
shutil.rmtree()	删除任意目录	rm -R
os.access()	判断对文件是否有指定的访问权限	
os.chmod()	修改文件或目录的权限	chmod
os.stat()	获取文件的基本信息	
os.system()	执行外部命令	